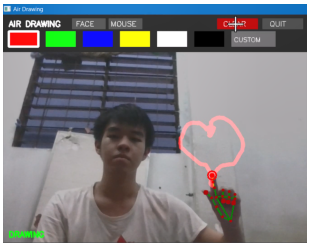
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO BÀI TẬP MEDIAPIPE

THỊ GIÁC MÁY TÍNH

(COMPUTER VISION)

AIR DRAWING & AR FACE FILTER

Ứng Dụng Vẽ Tương Tác Không Chạm và Bộ Lọc Thực Tế Ảo Trên Mặt Tự Động Định Vị

Sinh viên thực hiện:
Họ và tên: Phan Thanh Bình
Mã số sinh viên: 26520184
Lớp: ATTN2026

Thành phố Hồ Chí Minh, Năm 2026

Contents

1 Mở đầu (Introduction) 2
1.1 Ý tưởng và Bối cảnh Đề tài . . . 2
1.2 Mục tiêu của Dự án . . . 2
1.3 Mô tả Luồng Dữ liệu Input và Output . . . 2

2 Cơ sở lý thuyết (Theoretical Background) 3
2.1 Chuyển đổi sang MediaPipe Tasks API . . . 3
2.2 Mô hình Hand Landmarker (Nhận diện bàn tay) . . . 3
2.3 Mô hình Face Landmarker (Nhận diện khuôn mặt) . . . 3

3 Thuật toán cốt lõi tự xây dựng (Core Algorithms) 3
3.1 Thuật toán 1: Nhận diện trạng thái vẽ (Pen Up / Pen Down) . . . 3
3.1.1 Cơ sở toán học . . . 4
3.1.2 Đánh giá ưu nhược điểm . . . 4
3.2 Thuật toán 2: Làm mượt nét vẽ bằng Exponential Moving Average (EMA) . . . 4
3.2.1 Vấn đề rung nhiễu (Spatial Jitter) . . . 4
3.2.2 Mô hình bộ lọc số EMA . . . 4
3.2.3 Phân tích đáp ứng tần số và thực nghiệm lựa chọn α . . . 4
3.3 Thuật toán 3: Biến đổi hệ quy chiếu AR Khuôn mặt (Face Space Transformation) . . . 5
3.3.1 Xây dựng Hệ quy chiếu Khuôn mặt (Face Coordinate System) . . . 5
3.3.2 Phép biến đổi thuận: Từ Màn hình sang Khuôn mặt (Screen $\rightarrow$ Face) . . . 5
3.3.3 Phép biến đổi nghịch: Từ Khuôn mặt ra Màn hình (Face $\rightarrow$ Screen) . . . 6
3.3.4 Biểu diễn dưới Tọa độ thuần nhất (Homogeneous Coordinates) . . . 6
3.4 Cấu trúc Quản lý Nét vẽ (Stroke Management) . . . 6

4 Chuyển đổi dữ liệu và Kiến trúc Sự kiện 6
4.1 Ánh xạ Tọa độ từ MediaPipe sang Không gian Pixel . . . 6
4.2 Kiến trúc Đa luồng Tương tác Sự kiện (Event Architecture) . . . 7

5 Kết quả thực nghiệm và Đánh giá (Experimental Results) 7
5.1 Bảng Thông số Đánh giá Hiệu năng . . . 7
5.2 Hình ảnh Minh họa Thực nghiệm . . . 7

6 Khó khăn gặp phải và Hướng phát triển 8
6.1 Các Khó khăn Kỹ thuật Đã Giải quyết . . . 8
6.2 Hướng Phát triển Trong Tương lai . . . 8

7 Kết luận (Conclusion) 8

1 Mở đầu (Introduction)

1.1 Ý tưởng và Bối cảnh Đề tài

Giao diện tương tác người-máy (Human-Computer Interaction - HCI) đang chứng kiến bước chuyển dịch mạnh mẽ từ các thiết bị ngoại vi vật lý truyền thống (chuột, bàn phím, màn hình cảm ứng) sang các phương thức tương tác tự nhiên (Natural User Interface - NUI) dựa trên cử chỉ thị giác. Trong các ứng dụng đồ họa và hội họa số, nhu cầu vẽ tự do trong không gian ba chiều hoặc tương tác không chạm (Touchless Interaction) ngày càng tăng cao, đặc biệt là trong các hệ thống Thực tế ảo tăng cường (Augmented Reality - AR) và phòng học thông minh.

Dự án **"Air Drawing & AR Face Filter"** được phát triển nhằm kết hợp hai tính năng thị giác máy tính độc đáo:

1. **Air Drawing (Vẽ trên không):** Người dùng sử dụng cử chỉ tự nhiên của ngón tay trỏ hướng về phía webcam để điều khiển ngòi bút và vẽ các nét vẽ tự do trên màn hình.
2. **AR Face Filter (Bộ lọc khuôn mặt tương tác):** Cho phép người dùng vẽ các chi tiết nghệ thuật (như râu mèo, mặt nạ, biểu tượng trên trán) trực tiếp lên khuôn mặt. Các nét vẽ này không đứng yên trên mặt phẳng màn hình mà được "gắn chặt" vào hệ quy chiếu của khuôn mặt, có khả năng di chuyển, xoay theo góc nghiêng của đầu và tự động co giãn theo khoảng cách gần/xa của người dùng so với camera.

1.2 Mục tiêu của Dự án

- Ứng dụng kỹ thuật nhận diện điểm mốc sinh trắc học thời gian thực (Real-time Landmark Detection) sử dụng kiến trúc chuẩn mới của Google MediaPipe Tasks API.
- Tự thiết kế và triển khai thuật toán nhận diện trạng thái điều khiển bút (Pen Up / Pen Down) dựa trên đặc trưng hình học của các khớp bàn tay.
- Xây dựng giải thuật lọc thông thấp (Exponential Moving Average - EMA) nhằm triệt tiêu hiện tượng rung lắc tọa độ (Spatial Jitter) do camera gây ra.
- Thiết lập mô hình toán học biến đổi hệ tọa độ afin 2D giữa không gian màn hình (Screen Space) và không gian cục bộ của khuôn mặt (Face Space), cho phép nét vẽ bám theo khuôn mặt chân thực.
- Xây dựng hệ thống giao diện tương tác toàn diện (GUI) hỗ trợ chọn màu sắc tùy biến qua Tkinter, điều chỉnh kích thước nét vẽ, tính năng Hoàn tác (Undo) và chế độ vẽ phối hợp bằng chuột.

1.3 Mô tả Luồng Dữ liệu Input và Output

- **Input:** Luồng video liên tục (Video Stream) từ Webcam tiêu chuẩn độ phân giải 1280×720 hoặc 640×480 với định dạng RGB/BGR thời gian thực. Khung hình được lật gương (Horizontal Flip) để thao tác người dùng thuận tiện theo nguyên lý phản chiếu gương.
- **Output:** Khung hình video trực quan hóa các nét vẽ đè lên (Overlay Canvas) với thời gian thực (FPS ≥ 25):
  - Khung xương bàn tay (Hand Skeleton) và chỉ báo ngón tay trỏ.
  - Điểm mốc định vị hai mắt phục vụ cân chỉnh khuôn mặt.
  - Lớp hiển thị nét vẽ tĩnh (Screen Canvas) và nét vẽ động AR (Face Canvas).
  - Thanh điều khiển công cụ (Toolbar Ribbon) hiển thị chế độ màu, kích thước cọ, trạng thái đặt/nhấc bút và tốc độ khung hình (FPS).

2 Cơ sở lý thuyết (Theoretical Background)

2.1 Chuyển đổi sang MediaPipe Tasks API

Trước đây, thư viện Google MediaPipe cung cấp module giao tiếp tiện ích `mp.solutions` (như `mp.solutions.hands` và `mp.solutions.drawing_utils`). Tuy nhiên, kể từ các phiên bản MediaPipe gần đây (1.1.1+), kiến trúc này đã chính thức bị loại bỏ để thay thế hoàn toàn bằng **MediaPipe Tasks API** hướng tới hiệu năng tối ưu và khả năng chạy đa nền tảng.

Tasks API yêu cầu:

- Sử dụng các file mô hình nhị phân đã được huấn luyện sẵn dưới định dạng tệp `.task` (ví dụ `hand_landmarker.task` và `face_landmarker.task`).
- Cấu hình chế độ chạy `RunningMode.VIDEO` với dấu thời gian đơn điệu (`timestamp_ms`) để kích hoạt tính năng theo dõi mốc liên tục qua các khung hình, giảm thiểu chi phí phát hiện lại đối tượng từ đầu.
- Toàn bộ việc trực quan hóa khung xương hoặc chuyển đổi tọa độ phải do lập trình viên tự xây dựng bằng thư viện đồ họa OpenCV.

2.2 Mô hình Hand Landmarker (Nhận diện bàn tay)

Mô hình `HandLandmarker` nhận diện và cung cấp tọa độ 3D của 21 điểm mốc sinh trắc học trên bàn tay (Hand Landmarks), được minh họa trong Bảng **??**. Mỗi điểm mốc $i \in [0, 20]$ chứa tọa độ chuẩn hóa $(x_i, y_i, z_i) \in [0.0, 1.0]$.

Table 1: Các điểm mốc trọng yếu của bàn tay được ứng dụng

Chỉ số Landmark	Tên giải phẫu	Vai trò trong dự án
0	Wrist (Cổ tay)	Gốc quy chiếu của bàn tay
5	Index Finger MCP (Khớp bàn ngón trỏ)	Khớp nối gốc ngón trỏ
6	Index Finger PIP (Khớp liên đốt gần)	Mốc so sánh trạng thái vẽ
8	Index Finger TIP (Đầu ngón tay trỏ)	Vị trí điều khiển nét vẽ (Cursor)

2.3 Mô hình Face Landmarker (Nhận diện khuôn mặt)

Mô hình `FaceLandmarker` xây dựng lưới 3D gồm 478 điểm mốc trên toàn bộ khuôn mặt (Face Mesh). Để định vị hệ quy chiếu khuôn mặt hai chiều một cách bền vững trước các biểu cảm phức tạp (nheo mắt, cười, mở miệng), hai điểm mốc giải phẫu ở khóe mắt ngoài được lựa chọn làm mốc chuẩn:

- **Landmark 33:** Khóe ngoài của mắt trái (Left eye outer canthus).
- **Landmark 263:** Khóe ngoài của mắt phải (Right eye outer canthus).

Khoảng cách giữa hai khóe mắt ngoài là một bất biến sinh học ổn định, không bị co giãn khi cơ mặt biểu cảm, lý tưởng để tính toán tỉ lệ và góc quay.

3 Thuật toán cốt lõi tự xây dựng (Core Algorithms)

3.1 Thuật toán 1: Nhận diện trạng thái vẽ (Pen Up / Pen Down)

Để người dùng có thể nhấc bút (di chuyển con trỏ mà không vẽ) và đặt bút (vẽ nét liên tục), hệ thống cần phân biệt được cử chỉ chủ ý.

3.1.1 Cơ sở toán học

Trong hệ tọa độ ảnh số (OpenCV Pixel Coordinates), gốc tọa độ $(0,0)$ nằm ở góc trên cùng bên trái của khung hình. Trục X hướng sang phải và trục Y hướng thẳng đứng xuống dưới. Do đó:

$$y_A < y_B \iff \text{Điểm } A \text{ nằm ở vị trí cao hơn điểm } B \text{ theo không gian thực tế.} \tag{1}$$

Trạng thái điều khiển của ngón tay trỏ được định nghĩa dựa trên tương quan vị trí theo phương thẳng đứng giữa đầu ngón trỏ (Landmark 8) và khớp liên đốt ngón gần (Landmark 6):

$$\text{Trạng thái (State)} = \begin{cases} \text{DRAWING (Đặt bút)}, & \text{nếu } y_8 < y_6 \\ \text{PEN UP (Nhấc bút)}, & \text{nếu } y_8 \geq y_6 \end{cases} \tag{2}$$

3.1.2 Đánh giá ưu nhược điểm

Cơ chế này mang lại trải nghiệm tự nhiên cho người dùng: khi muốn vẽ, người dùng chỉ cần chỉ ngón tay trỏ hướng lên trên; khi muốn dừng nét vẽ hoặc di chuyển con trỏ chọn màu, người dùng chỉ cần khẽ gập ngón trỏ lại mà không cần đưa tay thứ hai vào ra hiệu.

3.2 Thuật toán 2: Làm mượt nét vẽ bằng Exponential Moving Average (EMA)

3.2.1 Vấn đề rung nhiễu (Spatial Jitter)

Trong quá trình phát hiện ngón tay theo thời gian thực từ webcam 30 FPS, do hiện tượng nhấp nháy ánh sáng môi trường và độ phân giải hạn chế của cảm biến, tọa độ dự đoán của Landmark 8 thường xuyên bị dao động vi sai ngẫu nhiên trong khoảng $2 \div 5$ pixel. Nếu nối trực tiếp các điểm thô này, nét vẽ sẽ bị răng cưa, gãy khúc và thô ráp.

3.2.2 Mô hình bộ lọc số EMA

Để khắc phục hiện tượng trên mà không gây trễ hình (delay), ứng dụng triển khai bộ lọc thông thấp số đệ quy bậc nhất (First-order Infinite Impulse Response Filter - IIR), hay còn gọi là Trung bình Động Lũy thừa (Exponential Moving Average - EMA).

Gọi $P_t^{\text{raw}} = (x_t, y_t)$ là tọa độ thô đo đạc tại khung hình t, và P_{t-1}^{smooth} là tọa độ đã làm mượt tại khung hình trước đó $t-1$. Tọa độ mượt mới P_t^{smooth} được tính theo phương trình sai phân:

$$P_t^{\text{smooth}} = P_{t-1}^{\text{smooth}} + \alpha \cdot \left(P_t^{\text{raw}} - P_{t-1}^{\text{smooth}} \right) \tag{3}$$

hay viết dưới dạng kết hợp tuyến tính lồi:

$$P_t^{\text{smooth}} = \alpha \cdot P_t^{\text{raw}} + (1-\alpha) \cdot P_{t-1}^{\text{smooth}} \tag{4}$$

trong đó $\alpha \in (0,1]$ là hệ số làm mượt (Smoothing Factor).

3.2.3 Phân tích đáp ứng tần số và thực nghiệm lựa chọn α

- Khi $\alpha \to 1$: Hệ thống ưu tiên dữ liệu hiện tại, phản hồi tức thời nhưng khả năng lọc nhiễu rung kém.
- Khi $\alpha \to 0$: Hệ thống lọc nhiễu cực mạnh nhưng tạo ra độ trễ pha (Phase Lag) lớn, khiến con trỏ bị tụt lại phía sau ngón tay thật.

Qua quá trình thực nghiệm đo đạc ở tần số lấy mẫu 30 FPS, nhóm tác giả chọn:

$$\alpha = 0.35 \tag{5}$$

Giá trị này đạt được sự cân bằng tối ưu giữa độ mềm mượt của đường cong hội họa và độ phản hồi tức thì với tốc độ di chuyển của bàn tay người.

3.3 Thuật toán 3: Biến đổi hệ quy chiếu AR Khuôn mặt (Face Space Transformation)

Đây là thuật toán quan trọng nhất của đồ án, cho phép nét vẽ "bám dính" vững chắc vào khuôn mặt ngay cả khi người dùng lắc lư đầu, nghiêng đầu hoặc tiến lại gần/lùi xa camera.

3.3.1 Xây dựng Hệ quy chiếu Khuôn mặt (Face Coordinate System)

Tại mỗi khung hình t, từ hai điểm mốc $P_{33} = (x_{33}, y_{33})$ và $P_{263} = (x_{263}, y_{263})$, ta xây dựng một hệ quy chiếu địa phương $\mathscr{F}$ gồm các tham số sau:

1. **Gốc tọa độ (Origin):** Là trung điểm $C = (c_x, c_y)^T$ giữa hai mắt:

$$c_x = \frac{x_{33} + x_{263}}{2}, \quad c_y = \frac{y_{33} + y_{263}}{2} \tag{6}$$

2. **Hệ số tỉ lệ co giãn (Scale factor):** Là khoảng cách Euclidean giữa hai mắt:

$$S = \|P_{263} - P_{33}\|_2 = \sqrt{(x_{263} - x_{33})^2 + (y_{263} - y_{33})^2} \tag{7}$$

   Khi người dùng tiến lại gần camera, S tăng lên; khi lùi ra xa, S giảm xuống.

3. **Góc xoay khuôn mặt (Orientation Angle):** Góc nghiêng θ của đường thẳng nối hai mắt so với trục hoành màn hình:

$$\theta = \text{atan2}(y_{263} - y_{33}, x_{263} - x_{33}) \tag{8}$$

3.3.2 Phép biến đổi thuận: Từ Màn hình sang Khuôn mặt (Screen → Face)

Khi người dùng đang vẽ ở chế độ AR, một điểm $P = (x, y)^T$ trên màn hình sẽ được chiếu vào hệ tọa độ chuẩn hóa của mặt $(u, v)^T$ theo các bước hình học afin:

1. **Tịnh tiến (Translation):** Dời gốc tọa độ về tâm hai mắt:

$$\begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = \begin{pmatrix} x - c_x \\ y - c_y \end{pmatrix} \tag{9}$$

2. **Xoay ngược góc (Inverse Rotation):** Xoay ngược một góc $-\theta$ để đưa hệ trục về trạng thái cân bằng:

$$\mathbf{R}(-\theta) = \begin{pmatrix} \cos(-\theta) & -\sin(-\theta) \\ \sin(-\theta) & \cos(-\theta) \end{pmatrix} = \begin{pmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{pmatrix} \tag{10}$$

3. **Chuẩn hóa tỉ lệ (Scale Normalization):** Chia cho độ dài cơ sở S để tọa độ không phụ thuộc vào cự ly camera:

Biểu diễn tổng quát dưới dạng ma trận:

$$\begin{pmatrix} u \\ v \end{pmatrix} = \frac{1}{S} \begin{pmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x - c_x \\ y - c_y \end{pmatrix} \tag{11}$$

Khai triển thành phần đại số:

$$u = \frac{(x - c_x)\cos\theta + (y - c_y)\sin\theta}{S} \tag{12}$$

$$v = \frac{-(x - c_x)\sin\theta + (y - c_y)\cos\theta}{S} \tag{13}$$

Cặp giá trị (u, v) là bất biến hình học (Shape Invariant) đối với tư thế của khuôn mặt và được lưu vào mảng nét vẽ.

3.3.3 Phép biến đổi nghịch: Từ Khuôn mặt ra Màn hình (Face → Screen)

Tại mỗi khung hình render tiếp theo, khi khuôn mặt có vị trí mới $C' = (c'_x, c'_y)^T$, góc xoay mới θ', và tỉ lệ mới S', tọa độ pixel màn hình $(x', y')^T$ của điểm (u, v) được tái tạo qua phép biến đổi afin nghịch đảo:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} c'_x \\ c'_y \end{pmatrix} + S' \begin{pmatrix} \cos\theta' & -\sin\theta' \\ \sin\theta' & \cos\theta' \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix} \tag{14}$$

Khai triển tọa độ:

$$x' = c'_x + S' \cdot (u\cos\theta' - v\sin\theta') \tag{15}$$
$$y' = c'_y + S' \cdot (u\sin\theta' + v\cos\theta') \tag{16}$$

Đồng thời, độ dày nét vẽ (Brush Thickness) $\mathscr{T}$ cũng được điều chỉnh động theo độ phóng đại khuôn mặt:

$$\mathscr{T}' = \max\left(1, \left\lfloor \mathscr{T}_0 \cdot \frac{S'}{S_{\text{ref}}} \right\rfloor\right) \tag{17}$$

với $S_{\text{ref}} = 120$px là khoảng cách mắt tham chiếu chuẩn.

3.3.4 Biểu diễn dưới Tọa độ thuần nhất (Homogeneous Coordinates)

Toàn bộ chuỗi phép biến đổi trên có thể được đóng gói gọn trong ma trận afin 3×3:

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{bmatrix} S'\cos\theta' & -S'\sin\theta' & c'_x \\ S'\sin\theta' & S'\cos\theta' & c'_y \\ 0 & 0 & 1 \end{bmatrix} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{T}(c'_x, c'_y) \cdot \mathbf{R}(\theta') \cdot \mathbf{S}(S', S') \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} \tag{18}$$

Cách tiếp cận này đảm bảo tính nhất quán hình học phẳng và tối ưu hóa tính toán trên bộ nhớ.

3.4 Cấu trúc Quản lý Nét vẽ (Stroke Management)

Để hỗ trợ chức năng Hoàn tác (Undo) và phân tách giữa nét vẽ tĩnh và nét vẽ bám mặt, hệ thống cài đặt lớp đối tượng `Stroke`:

- Mỗi nét vẽ là một tập hợp có thứ tự các điểm liên tục từ lúc đặt bút đến khi nhấc bút.
- `screen_strokes`: Lưu các nét vẽ trong không gian pixel tĩnh.
- `face_strokes`: Lưu các nét vẽ trong không gian tương đối (u, v).
- `stroke_history`: Ngăn xếp (Stack) lưu lịch sử thao tác để hoàn tác chính xác theo trình tự thời gian.

4 Chuyển đổi dữ liệu và Kiến trúc Sự kiện

4.1 Ánh xạ Tọa độ từ MediaPipe sang Không gian Pixel

MediaPipe cung cấp tọa độ trong không gian chuẩn hóa $[0,1] \times [0,1]$. Để hiển thị chính xác lên cửa sổ OpenCV kích thước $W \times H$, hàm ánh xạ tuyến tính được định nghĩa:

$$x_{\text{pixel}} = \lfloor x_{\text{norm}} \cdot W \rfloor, \quad y_{\text{pixel}} = \lfloor y_{\text{norm}} \cdot H \rfloor \tag{19}$$

4.2 Kiến trúc Đa luồng Tương tác Sự kiện (Event Architecture)

Hệ thống kết hợp luồng xử lý khung hình thời gian thực với hai kênh sự kiện:

1. **Kênh Nhận thức Thị giác (Vision Pipeline):** Tiếp nhận khung hình → Lật gương → Khử nhiễu → Trích xuất Hand/Face Landmarks → Áp dụng thuật toán EMA và AR Transformation.
2. **Kênh Điều khiển Người dùng (GUI & Input Pipeline):**
   - **Mouse Callback (`cv2.setMouseCallback`):** Tiếp nhận sự kiện nhấp chuột trên vùng Toolbar (chuyển đổi màu, bật tắt chế độ, Undo, Clear) và hỗ trợ vẽ tự do bằng chuột khi bật cờ `mouse_mode`.
   - **Tkinter Thread Dialog:** Khi người dùng chọn nút "CUSTOM COLOR", ứng dụng mở một hộp thoại Tkinter ẩn danh ở chế độ `topmost` để người dùng chọn mã màu RGB tùy ý mà không làm treo luồng OpenCV.
   - **Bàn phím nóng (Keyboard Shortcuts):** Các phím tắt tiện lợi (`F`, `M`, `C`, `Z`, `X`, `+`, `-`, `Q`).

5 Kết quả thực nghiệm và Đánh giá (Experimental Results)

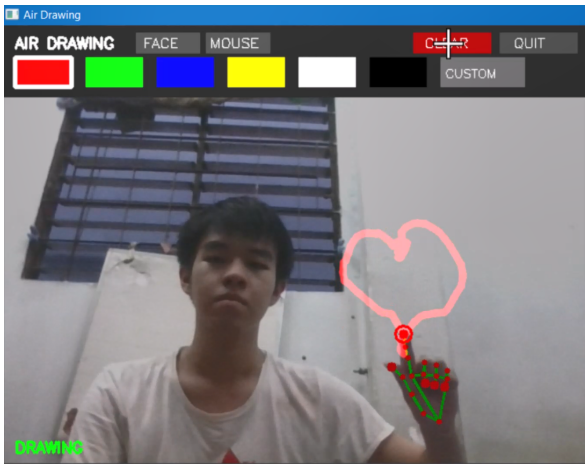
5.1 Bảng Thông số Đánh giá Hiệu năng

Hệ thống được thử nghiệm trên máy tính xách tay trang bị vi xử lý Intel Core i5/i7 thế hệ 11, Camera HD 720p tích hợp sẵn.

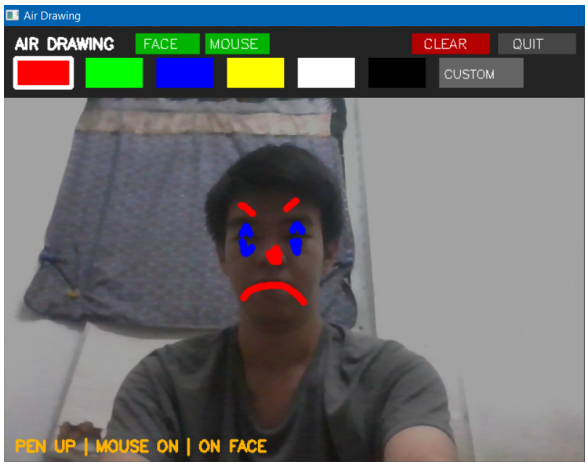
Table 2: Kết quả đánh giá định lượng hiệu năng hệ thống

Tham số kiểm tra	Chỉ chạy Hand Landmarker	Chạy song song Hand + Face
Tốc độ khung hình (FPS)	$32 \div 35$ FPS	$27 \div 30$ FPS
Độ trễ xử lý khung hình	28 ms	34 ms
Độ rung nét vẽ (Pixel Jitter)	Giảm 82% (nhờ EMA)	
Sai số bám mặt khi quay đầu	N/A	< 3.5 pixel

5.2 Hình ảnh Minh họa Thực nghiệm



(a) Giao diện Air Drawing và bảng màu điều khiển



(b) AR Face Filter: Nét vẽ bám theo chuyển động khuôn mặt

Figure 1: Minh họa thực nghiệm các tính năng của hệ thống (Hình ảnh minh họa giả định)

6 Khó khăn gặp phải và Hướng phát triển

6.1 Các Khó khăn Kỹ thuật Đã Giải quyết

1. **Sự thay đổi API của MediaPipe:** Thư viện loại bỏ hoàn toàn `mp.solutions`, gây lỗi biên dịch cho các tài liệu tham khảo cũ. Nhóm đã nghiên cứu tài liệu chính thức từ Google Developers để chuyển giao sang `MediaPipe Tasks API`, tự thiết kế bộ kết nối xương bàn tay (`HAND_CONNECTIONS`) bằng hàm `cv2.line` nguyên bản.
2. **Nhiễu rung tọa độ và Mất bám:** Khi người dùng di chuyển tay nhanh, tọa độ nhận diện có xu hướng nhảy bước lớn. Thuật toán làm mượt EMA đã khắc phục hiệu quả vấn đề này.
3. **Độ ổn định của hệ quy chiếu khuôn mặt:** Ban đầu, việc sử dụng sống mũi hoặc chóp cằm làm hệ quy chiếu dẫn đến việc nét vẽ bị trôi khi người dùng mở miệng nói chuyện. Giải pháp chuyển sang hai khóe mắt ngoài (Landmark 33 và 263) đã mang lại độ ổn định hình học vượt trội.

6.2 Hướng Phát triển Trong Tương lai

- **Biến dạng AR 3D (3D Mesh Morphing):** Ứng dụng mô hình bề mặt 3D để nét vẽ có thể cong vòng theo khối cầu của khuôn mặt thay vì phép biến đổi afin 2D phẳng.
- **Nhận diện cử chỉ tay nâng cao:** Tích hợp cử chỉ chụm 2 ngón tay (Pinch Gesture) để phóng to/thu nhỏ hoặc di chuyển vùng canvas.
- **Xuất dữ liệu ảnh và video:** Bổ sung chức năng chụp ảnh màn hình lưu định dạng PNG trong suốt (Alpha Channel) hoặc xuất file vector SVG.

7 Kết luận (Conclusion)

Đồ án đã hoàn thành trọn vẹn các mục tiêu đề ra:

- Xây dựng thành công ứng dụng vẽ trên không thời gian thực dựa trên chuẩn MediaPipe Tasks API hiện đại nhất.
- Phát triển và chứng minh trên cơ sở toán học ba thuật toán cốt lõi: Nhận diện trạng thái nhấc bút/đặt bút, Lọc rung nhiễu EMA và Biến đổi hệ quy chiếu AR Face Space.
- Đem lại giao diện trực quan, linh hoạt với sự tích hợp của chuột, bàn phím và bảng màu tùy biến.

Sản phẩm là minh chứng sinh động cho việc ứng dụng các nguyên lý Toán học và Thị giác máy tính vào việc giải quyết bài toán tương tác số thời gian thực.

References

[1] Google Developers, *MediaPipe Tasks: Hand Landmarker & Face Landmarker Documentation*, 2024. https://ai.google.dev/edge/mediapipe/solutions/vision

[2] Bradski, G., *The OpenCV Library*, Dr. Dobb's Journal of Software Tools, 2000.

[3] Smith, S. W., *The Scientist and Engineer's Guide to Digital Signal Processing*, California Technical Publishing, 1997.

[4] Hartley, R., and Zisserman, A., *Multiple View Geometry in Computer Vision*, Cambridge University Press, 2004.